

Assignment: Training Neural Networks

DSA 8401: Applied Machine Learning Master in Data Science and Analytics

1. Goals

This 3-week activity focuses on understanding how to design, train and improve Neural Networks on image data. You will first build your own fully connected architectures from scratch, and only afterwards compare them with a classical convolutional network.

- Build and train fully connected Neural Networks using Keras.
- Design your own architecture rather than reusing a published one.
- Compare different network architectures (depth versus width).
- Understand gradient problems during training.
- Understand the importance of weight initialization.
- Compare optimizers and training parameters.
- Apply regularization and dropout.
- Evaluate, save and restore a trained model.
- Benchmark your own architecture against a classical CNN and explain the difference.

2. Assignment Description

Use the MNIST handwritten digits dataset to build a Neural Network that classifies an image into one of ten digit classes (0–9). MNIST contains 70,000 grayscale images of 28×28 pixels, split into 60,000 training and 10,000 test images.

Dataset: load directly with `keras.datasets.mnist.load_data()`, or download from <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

Important: you must design and train your own architecture first. Classical convolutional architectures may only be introduced in Task 8, as a benchmark against the model you built yourself. Starting from a pre-built or pre-trained CNN is not accepted.

Required Tasks

1. Data Preparation

- Load MNIST and confirm the shapes: 60,000 training and 10,000 test images of 28×28 pixels.
- Visualise a sample of the images and check the class distribution across the ten digits.
- Scale pixel values from 0–255 to the [0, 1] range, and explain why input scaling matters for gradient-based training.
- Flatten each 28×28 image into a 784-dimensional vector for the fully connected models and keep the original 28×28×1 tensors for the CNN comparison in Task 8.
- Encode the labels appropriately: one-hot with categorical cross-entropy, or integer labels with sparse categorical cross-entropy.

- Hold out a validation set from the training images (for example 55,000 train / 5,000 validation) and leave the 10,000 test images untouched until final evaluation.

2. Your Own Network Architecture

- Design your own fully connected architecture: 784 input units, a softmax output layer with 10 units, and hidden layers of your choosing.
- Build and train networks with 1–5 hidden layers.
- Experiment with different numbers of neurons per layer.
- Compare a shallow/wide network with a deeper/narrower network at a roughly similar parameter count.
- Report the parameter count, training time and accuracy of each variant in a summary table.
- Analyse the effect of architecture on performance and overfitting.

3. Activation Functions & Gradients

- Compare at least two activation functions, including ReLU and one of Sigmoid/Tanh, keeping everything else fixed.
- Compare their training behaviour using loss/metric curves.
- Train a deliberately deep Sigmoid network (five or more hidden layers) and show evidence of vanishing gradients, for example per-layer gradient norms or a loss curve that stalls.
- Explain vanishing and exploding gradients and how activation functions and network depth can affect gradient flow.

4. Weight Initialization

- Compare at least two initialization methods, including an appropriate method such as Xavier/Glorot or He initialization.
- Include one run with all weights initialized to zero and report what happens.
- Analyse how initialization affects convergence and model performance.
- Explain why proper initialization is important and why initializing all weights to zero is problematic.

5. Optimisation

- Compare Adam, SGD with momentum, and one other optimizer such as RMSprop, Nadam or AdamW.
- Experiment with different learning rates (for example 0.1, 0.01, 0.001) and batch sizes (for example 32, 128, 512).
- Analyse their effect on convergence speed, training stability and final performance.

6. Regularization

- Apply dropout and/or L2 regularization, and optionally early stopping.
- Analyse whether they improve generalization and reduce overfitting.
- Comment on how much overfitting your best model actually shows on MNIST, and why.

7. Final Fully Connected Model

- Select the best fully connected model based on the experiments above.

- Document the architecture, activation functions, initialization, optimizer, learning rate and batch size.
- Evaluate the final model on the 10,000 test images: report accuracy, a confusion matrix, and examples of misclassified digits.
- Save and restore the model and demonstrate predictions with the restored model.

8. Comparison with a Classical CNN

Complete this task only after Task 7 is finished and documented.

- Implement a classical convolutional architecture, such as LeNet-5 or a simple Conv–Pool–Conv–Pool–Dense stack, trained on the $28 \times 28 \times 1$ images.
- Train it under a comparable budget to your best fully connected model (similar number of epochs and the same optimizer where reasonable).
- Compare accuracy, parameter count, training time and error patterns between the two models.
- Explain why convolution and pooling suit image data: local receptive fields, weight sharing and translation invariance, and what spatial information is lost when a 28×28 image is flattened into 784 independent inputs.
- State clearly what your own architecture achieved before the CNN was introduced, and how far the gap closes.

3. Required Analysis

For each major experiment, you must explain:

What did I change? What happened? Why did it happen? What did I learn?

Your analysis must specifically discuss:

- Effect of network architecture (depth versus width).
 - Gradient problems.
 - Importance of initialization.
 - Effect of activation functions.
 - Optimizer and learning-rate behaviour.
 - Overfitting and regularization.
 - Your own fully connected architecture versus the classical CNN on image data.
- Use appropriate tables and plots to support your analysis. Every experiment should appear in a summary table recording the configuration, the validation result and your observation.

4. Hand In

Students will submit a Jupyter Notebook containing:

- Python code.
- Model architectures, including `model.summary()` for each model discussed.
- Results, plots and comparison tables.
- Model evaluation on the held-out test set.
- Personal analysis and conclusions.

Set a random seed and report your library versions so results can be reproduced.